



Pyomo Cheat Sheet: Energy Optimization Edition

1. The Workflow

Optimization in Pyomo follows a strict 5-step recipe:

1. **Initialize** the Model container.
2. **Define Sets** (The "indices" like Time).
3. **Define Variables** (The "decisions" like Power, SOC).
4. **Define Constraints** (The "physics" limits).
5. **Define Objective** (The "goal" like Maximize Profit).
6. **Solve & Extract**.

2. Basic Setup

Python

```
import pyomo.environ as pyo

# Create the container
model = pyo.ConcreteModel()

# Define Time Index (e.g., 0 to 23 hours)
# 'initialize' takes any Python list or range
model.T = pyo.Set(initialize=range(24))
```

3. Defining Variables (The Decisions)

Variables are what the solver is allowed to change.

Syntax: `pyo.Var(Index, domain=...)`

Variable Type	Pyomo Domain	Use Case
Continuous	<code>pyo.Reals</code>	Temperatures, Pressures
Positive Only	<code>pyo.NonNegativeReals</code>	Power, Energy, Prices
Binary	<code>pyo.Binary</code>	On/Off, Charging/Discharging

Example:

Python

```
# A variable indexed by time T, must be positive
model.power = pyo.Var(model.T, domain=pyo.NonNegativeReals)

# A binary variable (0 or 1) indexed by time T
model.status = pyo.Var(model.T, domain=pyo.Binary)
```

4. Defining Constraints (The Physics)

Constraints limit the variables. In Pyomo, we usually define a **Python Function** first, then attach it to the model.

The Pattern:

Python

```
# Step 1: Write the Rule Function
def my_rule(model, t):
    # Returns an equation: Left_Side <= Right_Side
    # Example: Power cannot exceed Max Limit
    return model.power[t] <= 5.0

# Step 2: Attach to Model
# Note: We pass the function name 'my_rule', do not call it with ()
model.limit_constraint = pyo.Constraint(model.T, rule=my_rule)
```

Handling "Next Step" Logic (Dynamic Constraints):

Common in batteries (\$SOC_t\$ depends on \$SOC_{t-1}\$).

Python

```
def storage_logic(model, t):
    if t == 0:
        # Special case for the first hour
        return model.soc[t] == 50 # Start with 50 MWh
    else:
        # Standard logic linking current time to previous time
        return model.soc[t] == model.soc[t-1] + model.power[t]
```

5. The Objective Function (The Goal)

What are we trying to achieve?

Syntax: `pyo.Objective(expr=..., sense=...)`

Python

```
# Define the math expression
# maximize: (Price * Power) summed over all time steps
revenue_expr = sum(model.power[t] * price_list[t] for t in model.T)
```

```
# Attach to model
model.obj = pyo.Objective(expr=revenue_expr, sense=pyo.maximize)
```

Note: Use `sense=pyo.minimize` for cost minimization.

6. Solving & Results

Python

```
# 1. Select Solver (CBC is a great open-source option)
solver = pyo.SolverFactory('cbc')
```

```
# 2. Solve
results = solver.solve(model)
```

```
# 3. Extract Values (Important!)
# You cannot plot 'model.power[0]', you must extract the value first.
val = pyo.value(model.power[0])
```

Pro-Tips for BESS Modeling

1. **Efficiency Matters:** When charging, you buy **more** from the grid than ends up in the battery . When discharging, you sell **less** to the grid than leaves the battery.
 2. **Debugging:** If the solver says `Infeasible` , it means your constraints contradict each other (e.g., you forced SOC to be 10 but Capacity is 5).
-